

Reusable Markup with Snippets

Q21

SNIPPET RENDER TAG TEMPLATE SYNTAX REUSABILITY

INTERVIEW QUESTION

Question. How do you write a reusable chunk of markup you can use in several places?

ANSWER.

The mechanism — name the markup once, render it by name

- **Define** with `{#snippet}`; **declare** the block once with `{#snippet name()}` and close it with `{/snippet}`; everything between is ordinary Svelte markup that now lives in exactly one place.
- **Render** with `{@render}`; **stamp** it into the page with `{@render name()}`, calling it as many times and in as many spots as you need.
- **One source kills drift**: because the markup exists once, editing the snippet updates every spot that renders it; the silent inconsistency of five copy-pasted copies (edit four, miss the one hiding in an `{#if}` branch) becomes structurally impossible.

Parameters and scope — snippets behave like functions

- **Take parameters**: write `{#snippet figure(image)}` and each call site passes its own data with `{@render figure(product)}`, so one block serves many different items.
- **Defaults and destructuring**: parameters can have default values and be destructured exactly like a function's; the one thing you cannot use is a rest parameter.
- **Obey lexical scope**: a snippet reads any variable visible where it is declared, from the `<script>` tag or a surrounding `{#each}` block, with no passing required.
- **Can call themselves**: a snippet may render itself recursively, so countdown-style or nested markup is possible.

Snippets as component props — markup as a value

- **First-class values**: hand a snippet to a child as a prop with `<Table data={fruits} {header} {row} />` and let the child decide when and where to render it; a snippet declared directly between a component's tags becomes a prop automatically.
- **The children snippet**: any content between the tags that is not a named snippet becomes the special `children` snippet, rendered with `{@render children()}`; use `{@render children?. ()}` when it may be absent.
- **Replaces deprecated slots**: slots (Svelte 4) could not take parameters and were not values, so snippets fully replace them (see Lecture 21 for the long form, Q22 for typing them with the `Snippet` interface).

```

{#snippet figure(image)}
  <figure>
    <img src={image.src} alt={image.caption} />
    <figcaption>{image.caption}</figcaption>
  </figure>
{/snippet}

{#each products as product}
  {#if product.href}
    <a href={product.href}>{@render figure(product)}</a>
  {:else}
    {@render figure(product)}
  {/if}
{/each}

```

- ① **Summary.** A snippet is a named, reusable block of markup declared with `{#snippet}` and stamped in with `{@render}`; it takes parameters like a function, obeys lexical scope, and is a first-class value you can pass into components as a prop (where leftover content becomes `children`), collapsing every duplicated copy into one definition and fully replacing deprecated slots.

ONE DEFINITION • MANY RENDERS

{#SNIPPET}

figure(image)

```

<figure>
  <img src={image.src}>
  <figcaption>{image.caption}</figcaption>
</figure>

```

WRITE ONCE

{@RENDER}

```

figure(hero)
hero gallery

```

{@RENDER}

```

figure(cart)
recently viewed

```

{@RENDER}

```

figure(grid)
search results

```

edit the snippet once → every render site updates